

РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Программный эмулятор предназначен для учебной работы с небольшими программами на языке ассемблер. Пользователь вводит текст программы, компилирует его во внутреннее представление, запускает выполнение и наблюдает изменение регистров, флагов, памяти, стека и консоли.

Документ описывает основные окна программы, порядок работы, поддерживаемый синтаксис ассемблера, системные вызовы и типовые ошибки пользователя.

1. Назначение программы

Эмулятор решает учебную задачу: показывает связь между текстом ассемблерной программы и состоянием виртуальной машины. Он не запускает реальные исполняемые файлы Windows и не выполняет системные вызовы операционной системы. Все операции происходят внутри виртуальной модели.

- ввод и компиляция ассемблерного текста
- пошаговое и непрерывное выполнение программы
- просмотр регистров, флагов и счетчика команд EIP
- просмотр и изменение виртуальной памяти
- работа с консольным вводом и выводом
- настройка внешнего вида редактора, консоли и интерфейса

2. Основные окна программы

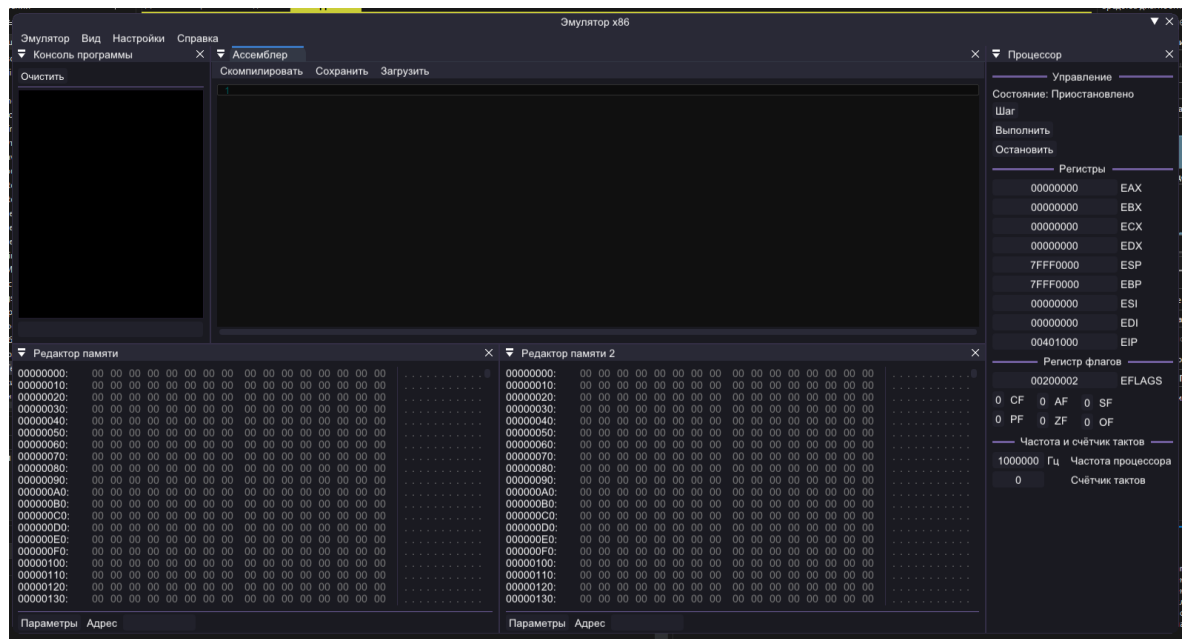


Рисунок 1 - Общий вид рабочей области эмулятора

2.1. Окно ассемблера

Окно ассемблера используется для ввода исходного кода. В нем доступны компиляция, сохранение и загрузка программы. Редактор поддерживает нумерацию строк, подсветку текущей строки, комментарии, строки, регистры, числа, метки и ошибки компиляции.

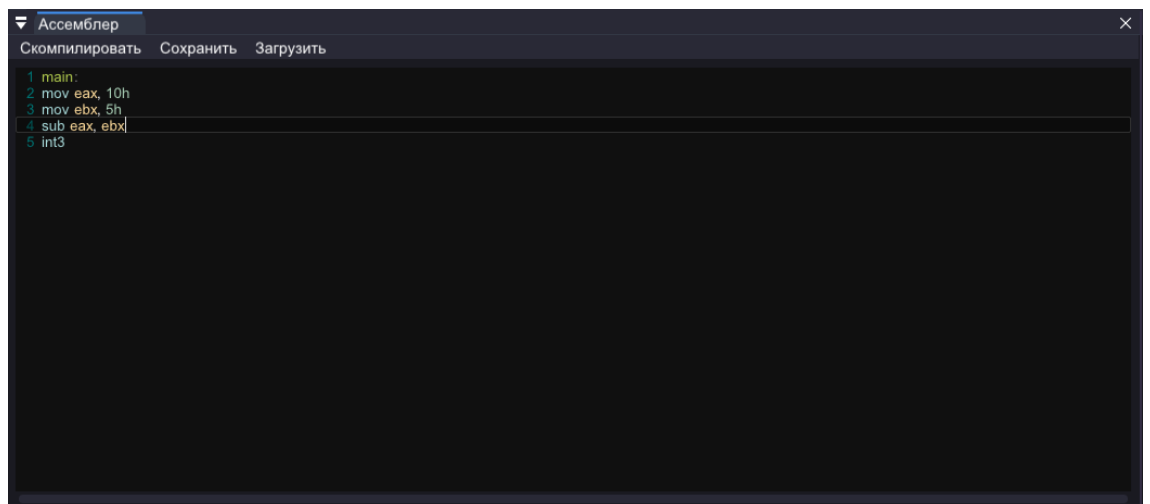


Рисунок 2 - Окно ассемблера с примером программы

2.2. Окно состояния процессора

Окно процессора показывает состояние выполнения, кнопки управления, регистры общего назначения, EIP, EFLAGS, отдельные флаги и счетчики. Кнопка шага выполняет одну инструкцию. Кнопка выполнения запускает программу до остановки, ошибки, ожидания ввода или int3.

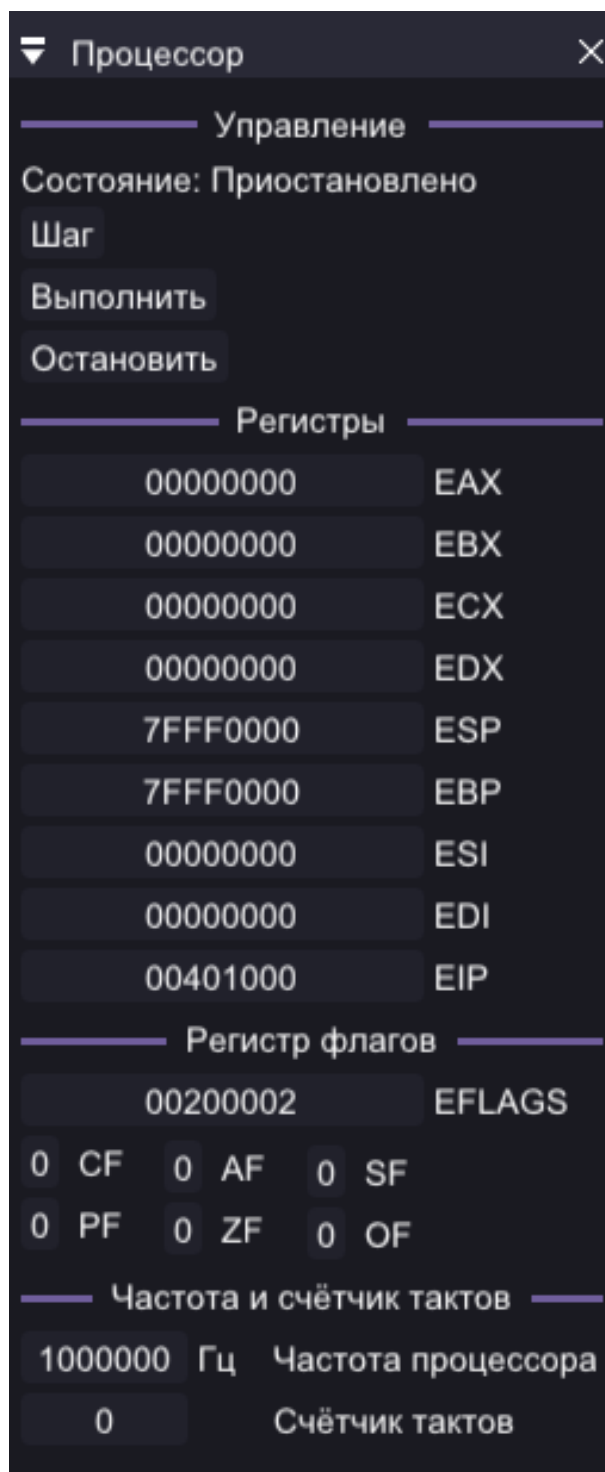


Рисунок 3 - Окно состояния процессора

Таблица 1 - Основные регистры процессора

Регистр	Назначение
EAX	Регистр общего назначения. Используется для результатов операций и номера syscall.
EBX	Регистр общего назначения для временных данных.
ECX	Регистр общего назначения. Часто применяется как счетчик.
EDX	Регистр общего назначения. Используется для адреса строки формата или буфера в syscall.
ESP	Указатель вершины стека.
EBP	Указатель кадра стека.
ESI	Регистр общего назначения, удобен для адресов источника.
EDI	Регистр общего назначения, удобен для адресов приемника.
EIP	Указатель текущей инструкции.

Таблица 2 - Основные флаги EFLAGS

Флаг	Назначение
CF	Флаг переноса для беззнаковой арифметики.
PF	Флаг четности младшего байта результата.
AF	Вспомогательный перенос между младшими тетрадами.
ZF	Флаг нуля. Устанавливается,

Флаг	Назначение
	если результат равен нулю.
SF	Флаг знака. Повторяет старший бит результата.
OF	Флаг переполнения для знаковой арифметики.

2.3. Редактор памяти

Редактор памяти показывает байты виртуальной памяти по адресам. Левая часть содержит адрес строки, центральная часть - значения байтов, правая часть - символьное представление. Через поле адреса можно перейти к нужной области памяти.

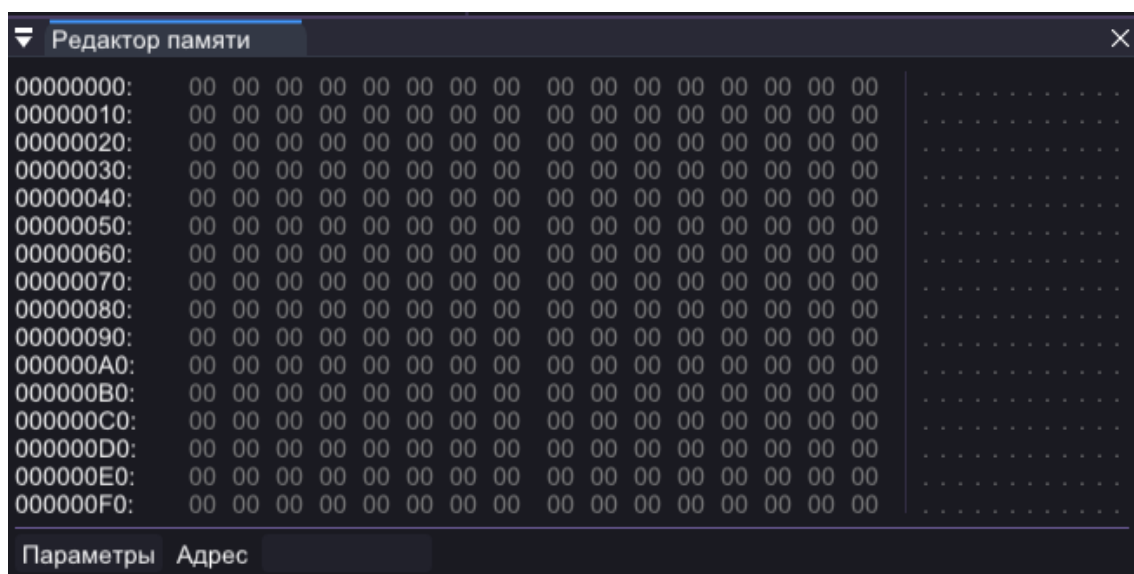


Рисунок 4 - Редактор виртуальной памяти

Область памяти используется для кода, данных, строк и стека. Начальное значение ESP размещает стек в верхней части доступной памяти. При выполнении push значение ESP уменьшается, при pop - увеличивается.

2.4. Консоль программы

Консоль используется для вывода текста и ввода строк. Если программа вызывает ввод, эмулятор переходит в состояние ожидания. В этом состоянии

кнопки шага и выполнения не продвигают EIP. Выйти из ожидания можно вводом строки или остановкой выполнения.

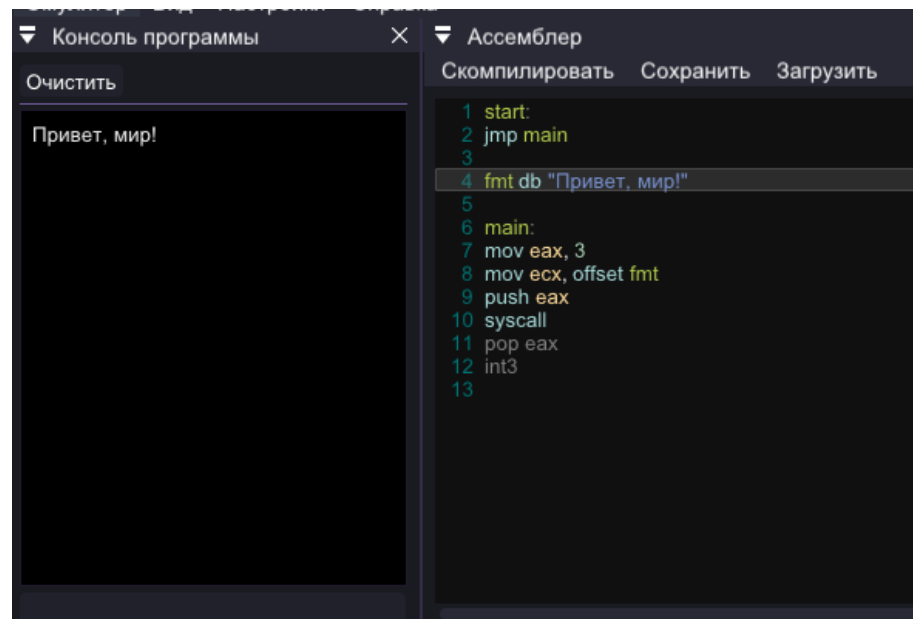


Рисунок 5 - Консоль и ассемблерная программа с выводом строки

2.5. Настройки

Окно настроек позволяет выбрать тему интерфейса, изменить шрифт, настроить отображение пробелов и табуляции, включить автосохранение и изменить цвета подсветки ассемблерного синтаксиса.

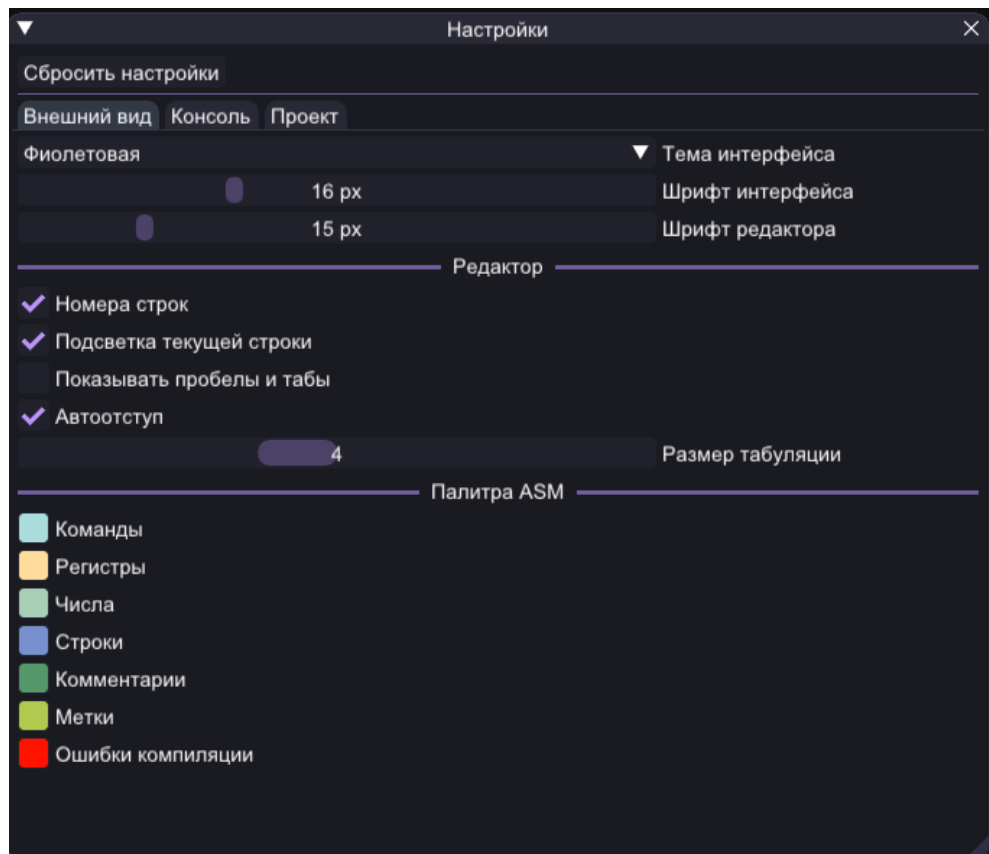


Рисунок 6 - Окно настроек приложения

3. Порядок работы

1. Открыть окно ассемблера.
2. Ввести исходный код программы.
3. Нажать кнопку компиляции.
4. Если появилось сообщение об ошибке, исправить строку с ошибкой и повторить компиляцию.
5. Открыть окна процессора, памяти и консоли.
6. Выполнить программу пошагово или запустить ее до остановки.
7. После каждого шага проверять EIP, регистры, флаги и измененные участки памяти.

4. Синтаксис ассемблерной программы

Программа состоит из меток, инструкций, операндов, директив данных и комментариев. Метка записывается с двоеточием. Комментарий начинается с

символа точки с запятой. Строки заключаются в двойные кавычки и сохраняются в памяти в кодировке UTF-8.

Таблица 3 - Элементы синтаксиса

Элемент	Пример	Описание
Метка	start:	Именованный адрес в коде или данных.
Инструкция	mov eax, 10h	Команда, выполняемая процессором.
Комментарий	; пояснение	Текст для пользователя, не выполняется.
Строка	msg db "Привет!", 0	Последовательность UTF-8 байтов в памяти.
32-битное значение	value dd 123	Директива размещения двойного слова.
Адрес метки	offset msg	Получение адреса строки или переменной.

Листинг 1 - Минимальная программа вывода строки

```
start:
    jmp main

msg db "Привет!", 0

main:
    mov eax, 1
    mov edx, offset msg
    syscall
    int3
```


5. Системные вызовы syscall

Инструкция `syscall` обращается к функциям консоли эмулятора. Номер функции передается через `EAX`. Регистр `EDX` обычно содержит адрес строки, буфера или строки формата. Для форматированных вызовов дополнительные параметры передаются через стек.

Таблица 4 - Системные вызовы

EAX	Название	Параметры	Назначение
1	write	EDX - адрес строки	Выводит строку UTF-8 до завершающего нулевого байта.
2	read	EDX - адрес буфера	Ожидает ввод строки и записывает ее в память.
3	printf	EDX - адрес формата, параметры - в стеке	Форматированный вывод.
4	scanf	EDX - адрес формата, адреса приемников - в стеке	Форматированный ввод.

Для `printf` параметры помещаются в стек справа налево. После `syscall` программист сам восстанавливает `ESP` командой `add esp, N`, где `N` - суммарный размер параметров. Для `scanf` в стек передаются адреса переменных или буферов.

Таблица 5 - Спецификаторы printf и scanf

Спецификатор	Printf	Scanf
--------------	--------	-------

Спецификатор	Printf	Scanf
%d	Знаковое десятичное число	Чтение знакового числа
%u	Беззнаковое десятичное число	Чтение беззнакового числа
%x	Шестнадцатеричное число	Чтение шестнадцатеричного числа
%c	Один символ	Чтение одного символа
%s	Строка UTF-8 по адресу	Чтение строки UTF-8 в буфер
%%	Символ процента	Символ процента в формате

Листинг 2 - Форматированный вывод

```
fmt db "eax=%d, ebx=%u, hex=%x", 0
```

```
main:
```

```
    push ebx
    push ebx
    push eax
    mov eax, 3
    mov edx, offset fmt
    syscall
    add esp, 12
    int3
```

6. Справочник команд

В таблице приведены команды и директивы, поддерживаемые учебным эмулятором. Набор ориентирован на лабораторные работы и демонстрацию базовых механизмов процессора.

Таблица 6 - Поддерживаемые команды и директивы

Команда	Группа	Назначение
mov dst, src	Передача данных	Копирует значение из источника в приемник. Операнды: регистр, память, непосредственное число.
lea dst, src	Передача данных	Вычисляет адрес операнда и записывает его в регистр. Удобна для работы с метками и буферами.
xchg op1, op2	Передача данных	Меняет значения двух операндов местами.
add dst, src	Арифметика	Складывает операнды, результат записывает в dst, обновляет флаги.
sub dst, src	Арифметика	Вычитает src из dst, результат записывает в dst, обновляет флаги.
inc op	Арифметика	Увеличивает операнд на единицу.
dec op	Арифметика	Уменьшает операнд на единицу.
neg op	Арифметика	Меняет знак операнда через дополнительный код.
mul src	Умножение	Беззнаковое умножение. Результат размещается в регистрах модели процессора.

Команда	Группа	Назначение
imul src	Умножение	Знаковое умножение. Используется для чисел в дополнительном коде.
div src	Деление	Беззнаковое деление. Деление на ноль переводит выполнение в состояние ошибки.
idiv src	Деление	Знаковое деление. Требуется корректной подготовки делимого и делителя.
and dst, src	Логика	Побитовое И. Обновляет флаги результата.
or dst, src	Логика	Побитовое ИЛИ. Обновляет флаги результата.
xor dst, src	Логика	Побитовое исключающее ИЛИ. Часто используется для обнуления регистра.
not op	Логика	Инвертирует все биты операнда.
test op1, op2	Логика	Выполняет побитовое И только для установки флагов, результат не сохраняет.
shl op, n	Сдвиги	Логический сдвиг влево.
shr op, n	Сдвиги	Логический сдвиг

Команда	Группа	Назначение
		вправо.
sar op, n	Сдвиги	Арифметический сдвиг вправо с сохранением знака.
cmp op1, op2	Сравнение	Выполняет вычитание без сохранения результата, изменяет флаги для условных переходов.
clc	Флаги	Сбрасывает флаг переноса CF.
stc	Флаги	Устанавливает флаг переноса CF.
push op	Стек	Уменьшает ESP и записывает значение на вершину стека.
pop op	Стек	Читает значение с вершины стека и увеличивает ESP.
call label	Подпрограммы	Сохраняет адрес возврата в стеке и передает управление метке.
ret	Подпрограммы	Возвращает управление по адресу, лежащему на вершине стека.
jmp label	Переходы	Безусловно изменяет EIP.
je label	Переходы	Переходит, если ZF = 1.

Команда	Группа	Назначение
jne label	Переходы	Переходит, если $ZF = 0$.
ja label	Переходы	Беззнаковый переход, если $op1 > op2$ после <i>сmp</i> .
jbe label	Переходы	Беззнаковый переход, если $op1 \leq op2$ после <i>сmp</i> .
jb label	Переходы	Беззнаковый переход, если $op1 < op2$ после <i>сmp</i> .
jnb label	Переходы	Беззнаковый переход, если $op1 \geq op2$ после <i>сmp</i> .
jl label	Переходы	Знаковый переход, если $op1 < op2$ после <i>сmp</i> .
jle label	Переходы	Знаковый переход, если $op1 \leq op2$ после <i>сmp</i> .
jg label	Переходы	Знаковый переход, если $op1 > op2$ после <i>сmp</i> .
jge label	Переходы	Знаковый переход, если $op1 \geq op2$ после <i>сmp</i> .
jo label	Переходы	Переходит, если установлен OF.
jno label	Переходы	Переходит, если $OF = 0$.
js label	Переходы	Переходит, если

Команда	Группа	Назначение
		установлен SF.
jns label	Переходы	Переходит, если SF = 0.
nop	Служебные	Не выполняет действий, используется для пустого шага.
int3	Служебные	Останавливает выполнение программы как программная точка останова.
syscall	Ввод-вывод	Вызывает функцию консоли. Номер функции передается через EAX.
db	Данные	Размещает байты и строки UTF-8 в памяти.
dd	Данные	Размещает 32-битные значения в памяти.
offset label	Данные	Возвращает адрес метки. Применяется для строк, буферов и переменных.

7. Типовые ошибки пользователя

Таблица 7 - Типовые ошибки и способы исправления

Ситуация	Причина	Что проверить
Неизвестная инструкция	Опечатка или неподдерживаемая	Сравнить мнемонику со

Ситуация	Причина	Что проверить
	команда	справочником команд.
Неверный операнд	Команда получила операнд неподходящего типа	Проверить регистр, память, число и порядок операндов.
Переход не выполняется	Флаги не соответствуют условию	Посмотреть ZF, SF, OF, CF после cmp или test.
Строка выводится неверно	Нет завершающего нуля или неверный адрес	Проверить директиву db и значение offset.
После printf ломается стек	Параметры не убраны из стека	Добавить add esp, N после syscall.
Ввод не срабатывает заранее	Программа еще не находится в состоянии ожидания	Сначала дойти до read или scanf, затем вводить строку.
Деление остановило программу	Делитель равен нулю	Проверить операнд перед div или idiv.

8. Рекомендации по учебной работе

- начинать проверку программы с пошагового режима
- после арифметики смотреть не только результат, но и флаги
- при работе со стеком контролировать ESP до push и после pop
- для строк проверять адрес, байты UTF-8 и завершающий нулевой байт
- после форматированных syscall всегда восстанавливать ESP
- использовать int3 как понятную точку завершения учебной программы

9. Подробное описание сложных инструкций

Некоторые инструкции используют неявные регистры. Это особенно важно для умножения, деления, сдвигов, переходов и вызовов подпрограмм.

Если учитывать только записанные в строке операнды, поведение таких команд может выглядеть неожиданным.

9.1. Умножение mul и imul

Команды mul и imul имеют один явный операнд. Второй множитель всегда берется из регистра EAX. После выполнения 64-битный результат записывается в пару EDX:EAX: младшие 32 бита попадают в EAX, старшие 32 бита - в EDX.

Таблица 8 - Правила работы mul и imul

Команда	Тип чисел	Исходные данные	Результат	Флаги
mul src	Беззнаковые	EAX * src	EDX:EAX = 64-битное произведение	CF и OF равны 1, если EDX не равен 0
imul src	Знаковые	(int)EAX * (int)src	EDX:EAX = 64-битное произведение	CF и OF равны 1, если результат не помещается в int32

Операнд src должен быть регистром или значением из памяти. Непосредственное число без квадратных скобок для mul, imul, div и idiv не используется. Если нужно умножить на константу, сначала поместите ее в регистр или в память.

Листинг 3 - Пример использования mul и imul

main:

```

mov eax, 6
mov ebx, 7
mul ebx    // EDX:EAX = 42

mov eax, 6
mov ebx, 0
sub ebx, 7 // EBX = -7
imul ebx   // EDX:EAX = -42

```

int3

9.2. Деление `div` и `idiv`

Команды `div` и `idiv` делят не только `EAX`. Делимое всегда берется из пары `EDX:EAX`. В `EDX` находится старшая часть делимого, в `EAX` - младшая. После выполнения частное записывается в `EAX`, остаток - в `EDX`.

Таблица 9 - Правила работы `div` и `idiv`

Команда	Тип чисел	Делимое	Делитель	Результат
<code>div src</code>	Беззнаковые	<code>EDX:EAX</code>	<code>src</code>	<code>EAX</code> = частное, <code>EDX</code> = остаток
<code>idiv src</code>	Знаковые	<code>EDX:EAX</code>	<code>src</code>	<code>EAX</code> = частное, <code>EDX</code> = остаток

Для обычного 32-битного беззнакового деления перед `div` чаще всего нужно записать 0 в `EDX`. Для отрицательного знакового делимого перед `idiv` старшая часть должна содержать знаковое расширение: для отрицательного числа обычно `EDX = FFFFFFFFh`, для положительного `EDX = 0`.

Деление останавливается с ошибкой, если делитель равен нулю или если частное не помещается в 32 бита. Поэтому перед `div` и `idiv` стоит отдельно проверять делитель.

Листинг 4 - Пример использования `div` и `idiv`

```
main:
    mov edx, 0
    mov eax, 100
    mov ebx, 7
    div ebx    // EAX = 14, EDX = 2

    mov edx, 0
    sub edx, 1    // EDX = FFFFFFFFh
    mov eax, 0
```

```

sub eax, 100 // EAX = -100
mov ebx, 7
idiv ebx     // EAX = -14, EDX = -2
int3

```

9.3. Сдвиги shl, shr и sar

Команды сдвига используют два операнда: изменяемое значение и количество битов. Эмулятор учитывает только младшие 5 бит счетчика сдвига, поэтому фактическое количество находится в диапазоне от 0 до 31.

Таблица 10 - Правила работы команд сдвига

Команда	Действие	Особенность
shl dst, count	Сдвигает биты влево	Освободившиеся младшие биты заполняются нулями
shr dst, count	Логически сдвигает вправо	Старшие биты заполняются нулями
sar dst, count	Арифметически сдвигает вправо	Старший знаковый бит сохраняется

9.4. Сравнение, test и условные переходы

Команды cmp и test не сохраняют вычисленный результат в операндах. Их задача - подготовить флаги. После этого условные переходы читают ZF, CF, SF и OF и решают, нужно ли менять EIP.

Таблица 11 - Связь переходов с флагами

Переход	Условие	Тип сравнения
je	ZF = 1	Равно
jne	ZF = 0	Не равно
ja	CF = 0 и ZF = 0	Беззнаковое больше

Переход	Условие	Тип сравнения
jbe	CF = 1 или ZF = 1	Беззнаковое меньше или равно
jl	SF != OF	Знаковое меньше
jge	SF = OF	Знаковое больше или равно
jg	ZF = 0 и SF = OF	Знаковое больше
jle	ZF = 1 или SF != OF	Знаковое меньше или равно
jo / jno	OF = 1 / OF = 0	Проверка переполнения
js / jns	SF = 1 / SF = 0	Проверка знака

10. Внутренний байткод эмулятора

После компиляции ассемблерный текст превращается во внутренний байткод эмулятора. Этот байткод похож на машинный код только по идее: сначала идет байт opcode, затем байты операндов. Это не настоящий x86-код, а учебный формат проекта.

Общий формат инструкции: opcode, затем первый операнд, затем второй операнд, если он есть. Длина операнда зависит от его типа. Регистр занимает 1 байт, непосредственное 32-битное значение занимает 5 байт: код типа 0Ah и четыре байта числа в little-endian.

Таблица 12 - Байткоды opcode

Мнемоника	Opcode, hex	Группа	Описание
nop	90	Служебная	Пустая инструкция.
or	09	Логика	Побитовое ИЛИ.
syscall	0F	Ввод-вывод	Вызов

Мнемоника	Opcode, hex	Группа	Описание
			функции консоли эмулятора.
and	21	Логика	Побитовое И.
xor	31	Логика	Побитовое исключающее ИЛИ.
cmp	39	Сравнение	Сравнение без сохранения результата.
push	50	Стек	Запись значения в стек.
pop	58	Стек	Чтение значения из стека.
jo	70	Переход	Переход при $OF = 1$.
jno	71	Переход	Переход при $OF = 0$.
jb	72	Переход	Беззнаковый переход ниже.
jnb	73	Переход	Беззнаковый переход не ниже.
je	74	Переход	Переход при $ZF = 1$.
jne	75	Переход	Переход при $ZF = 0$.
jbe	76	Переход	Беззнаковый переход ниже или равно.
ja	77	Переход	Беззнаковый переход выше.
js	78	Переход	Переход при $SF = 1$.
jns	79	Переход	Переход при $SF = 0$.
jl	7C	Переход	Знаковый

Мнемоника	Opcode, hex	Группа	Описание
			переход меньше.
jge	7D	Переход	Знаковый переход больше или равно.
jle	7E	Переход	Знаковый переход меньше или равно.
jg	7F	Переход	Знаковый переход больше.
sub	81	Арифметика	Вычитание.
add	83	Арифметика	Сложение.
xchg	87	Передача данных	Обмен значений.
lea	8D	Адресация	Получение эффективного адреса.
test	85	Логика	Проверка битов без записи результата.
mov	8B	Передача данных	Копирование значения.
imul	AF	Умножение	Знаковое умножение EAX на операнд.
ret	C3	Подпрограммы	Возврат из подпрограммы.
shl	D1	Сдвиг	Логический сдвиг влево.
shr	D2	Сдвиг	Логический сдвиг вправо.
sar	D3	Сдвиг	Арифметический сдвиг вправо.
call	E8	Подпрограммы	Вызов подпрограммы.
clc	F0	Флаги	Сброс CF.

Мнемоника	Opcode, hex	Группа	Описание
stc	F1	Флаги	Установка CF.
neg	F5	Арифметика	Смена знака.
not	F6	Логика	Инверсия битов.
mul	F7	Умножение	Беззнаковое умножение EAX на операнд.
div	F8	Деление	Беззнаковое деление EDX:EAX на операнд.
idiv	F9	Деление	Знаковое деление EDX:EAX на операнд.
inc	FE	Арифметика	Увеличение на 1.
dec	FF	Арифметика	Уменьшение на 1.
int3	CC	Остановка	Программная точка остановки.
jmp	E9	Переход	Безусловный переход.

Таблица 13 - Основные байткоды операндов

Операнд	Байт, hex	Описание
eax	01	Регистр EAX
ebx	02	Регистр EBX
ecx	03	Регистр ECX
edx	04	Регистр EDX
esp	05	Регистр ESP
ebp	06	Регистр EBP
esi	07	Регистр ESI

Операнд	Байт, hex	Описание
edi	08	Регистр EDI
eip	09	Регистр EIP. Прямая запись как обычный операнд ограничена проверками декодера.
rawdata	0A + 4 байта	Непосредственное число, адрес метки или результат offset. Число хранится в little-endian.

Таблица 14 - Старшая тетрада байта операнда

Старшая тетрада	Форма	Пример	Дополнитель ные байты
0	Прямое обращение	eax	Нет
1	Чтение памяти по адресу	[eax], [400000h]	Для rawdata добавляются 4 байта адреса
2	Чтение по base + imm8	Внутренний вариант	1 байт смещения
3	Чтение по base * imm8	Внутренний вариант	1 байт множителя
4	Чтение по base + imm32	[eax+4]	4 байта смещения
5	Чтение по base * imm32	[eax*4]	4 байта множителя
6	Чтение по base + reg	[eax+ebx]	1 байт второго регистра
7	Чтение по base * reg	[eax*ebx]	1 байт второго регистра

Итоговый байт операнда собирается так: старшая тетрада задает способ адресации, младшая тетрада задает основной тип операнда. Например, `eax` имеет байт `01h`, `[eax]` имеет байт `11h`, `[eax+4]` имеет байт `41h` и четыре дополнительных байта смещения.

Таблица 15 - Примеры кодирования операндов

Запись	Байты	Пояснение
<code>eax</code>	<code>01</code>	Прямое обращение к EAX
<code>10h</code>	<code>0A 10 00 00 00</code>	Непосредственное значение <code>00000010h</code>
<code>offset msg</code>	<code>0A xx xx xx xx</code>	Адрес метки <code>msg</code> в little-endian
<code>[eax]</code>	<code>11</code>	Память по адресу из EAX
<code>[400000h]</code>	<code>1A 00 00 40 00</code>	Память по непосредственному адресу <code>00400000h</code>
<code>[eax+4]</code>	<code>41 04 00 00 00</code>	Память по адресу <code>EAX + 4</code>
<code>[eax+ebx]</code>	<code>61 02</code>	Память по адресу <code>EAX + EBX</code>
<code>[eax*4]</code>	<code>51 04 00 00 00</code>	Память по адресу <code>EAX * 4</code>

Таблица 16 - Примеры байткода инструкций

Ассемблер	Байты	Разбор
<code>mov eax, 10h</code>	<code>8B 01 0A 10 00 00 00</code>	<code>8B</code> - <code>mov</code> , <code>01</code> - <code>eax</code> , <code>0A...</code> - число <code>10h</code>
<code>add eax, ebx</code>	<code>83 01 02</code>	<code>83</code> - <code>add</code> , <code>01</code> - <code>eax</code> , <code>02</code> - <code>ebx</code>

Ассемблер	Байты	Разбор
mul ebx	F7 02	F7 - mul, 02 - ebx
div [400000h]	F8 1A 00 00 40 00	F8 - div, 1A... - память по адресу 00400000h
push eax	50 01	50 - push, 01 - eax
pop ebx	58 02	58 - pop, 02 - ebx
int3	CC	CC - программная остановка
syscall	0F	0F - вызов функции консоли